

Fitness Tracker Mobile App

PRODUCTION READY

React Native Machine Coding Assignment • Client Technical Submission

1. Instructions to Run the Project

Follow these steps to clone, configure, and launch the mobile application on an iOS Simulator or Android Emulator.

Step 1: Clone Repository & Install Dependencies

```
# Clone the repository git clone https://github.com/MalkeetBhangu/fitnesstracker.git cd
fitnesstracker # Install project dependencies npm install
```

Running on iOS (macOS)

```
# 1. Install CocoaPods native dependencies cd ios && pod install &&
cd .. # 2. Start Metro bundler in a terminal npm start # 3. Launch iOS
Simulator (iPhone 15/16) npm run ios
```

Running on Android

```
# 1. Ensure Android Emulator is running # (or physical device
connected via adb) # 2. Start Metro bundler npm start # 3. Launch Android
app build npm run android
```

Code Verification & Type Checking

```
# Run TypeScript compiler check (0 errors verified) npx tsc --noEmit # Run ESLint
quality checks npm run lint
```

2. Setup & Configuration Instructions

ENVIRONMENT / TOOL	REQUIRED VERSION	CONFIGURATION NOTES
Node.js	≥ 22.11.0 (LTS)	Managed via NVM or native installation. Tested on Node v22.14.0.
React Native / React	RN 0.87.1 • React 19.2.3	React Native Community CLI configured with Hermes engine enabled.
iOS Environment	Xcode ≥ 15.0 • CocoaPods ≥ 1.14	Deployment target iOS 15.1+. Run <code>pod install</code> in the <code>ios/</code> folder.
Android Environment	JDK 17 • Android SDK 34+	Ensure <code>ANDROID_HOME</code> is set in shell profile and platform-tools in <code>PATH</code> .
Asset Transformer	<code>react-native-svg-transformer</code>	Configured in <code>metro.config.js</code> for direct SVG component imports.
Path Aliases	TypeScript <code>paths</code> mapping	Configured in <code>tsconfig.json</code> with <code>@src/*</code> pointing directly to <code>src/*</code> .

3. Architecture & Key Technical Decisions

1. Polymorphic Section Feed Model (Virtualization & Performance)

The Progress screen uses a **Polymorphic Section Architecture** (`Section[]`) managed via `transformProgressToSections` . Rather than rendering individual hardcoded scroll views, server/mock responses are decoupled and transformed into strongly typed discriminated union items:

- **CurrentWeightCard** • **GoalCard** • **WeightChartCard** • **BmiCard** • **RecentLogsCard** • **WalkthroughCard**

This allows single-pass virtualization, recycling, smooth 60fps scrolling, and effortless addition/reordering of dashboard cards.

2. State Management & Zero-Latency Persistence (Zustand + MMKV)

User preferences (including selected language code) are managed through **Zustand** and paired directly with **MMKV** (`react-native-mmkv`) via Zustand's `persist` middleware. Unlike asynchronous `AsyncStorage` , MMKV offers synchronous C++ native bindings, eliminating UI flashes or race conditions during app rehydration.

3. Zero-Hardcoded-Strings Localization Pipeline (i18n)

Strict architectural rule enforced across the codebase: **100% of user-facing strings, units (`kg` , `cm`), months, and days** reside in `en.json` and `de.json` . The custom `getTexts(languageCode)` selector provides type-safe reactive updates whenever a user toggles languages via the header options modal (`...`).

4. Custom SVG Graphics & Precision Visualizations

Interactive graphics are built from scratch using `react-native-svg` :

- **Interactive Weight Chart**: Smooth cubic bezier curve with gradient fill, touchable active points, and dynamic speech bubble tooltip with notch pointer.
- **BMI Gauge**: 4-band colored gauge (Underweight, Normal, Overweight, Obese) with dynamic geometric pin placement calculated directly from current BMI values.

5. Design Tokens & Responsive Dimensions

Centralized design token system in `Colors.ts` (strict RGBA color palette) and `StyleHelper.ts` (`getHeight` , `getWidth`) ensuring pixel-perfect fidelity across all iOS and Android device screen densities.

4. Scope Note & Future Enhancements (4-Hour Limit)

Within the allotted 4-hour machine coding timeframe, the primary focus was on completing all core Progress Dashboard cards, pixel-perfect UI fidelity, interactive SVG charts and gauges, custom bottom tab navigation, persistent multi-language localization (EN/DE), and full TypeScript type safety (0 errors).

Items Scoped for Subsequent Iteration:

- **Embedded Native Video Playback:** The Walkthrough Carousel contains full thumbnail layouts, play button overlays, tag badges, and metadata; integrating a full native video playback player (e.g. `react-native-video` modal player) was deferred to future sprint work.
- **Full CRUD Mutation Forms:** Interactive edit buttons on Weight, Goal, and BMI cards are connected to event handlers; connecting them to editable bottom-sheet modals with backend API write mutations was scheduled for the next phase.
- **Detox E2E Test Suite:** TypeScript compilation (`tsc --noEmit`) and ESLint are fully passing with 0 errors; full automated end-to-end Detox test suites and Jest component snapshot tests were reserved for future releases.